

TRABALHO PRATICO - FASE IV

Loja Online

Relatorio Tecnico de Compressao

Arquitetura: MVC + DAO em Java, com interface web local

Persistencia: arquivos binarios com cabecalho, lapide e indices persistentes

Evolucao: backup compactado em arquivo unico usando Huffman e LZW

A Fase IV compacta todos os arquivos `.db` usados pelo aplicativo em um unico backup. O sistema primeiro empacota os dados e indices em um fluxo binario completo e depois aplica Huffman ou LZW sobre esse arquivo logico, preservando o funcionamento das etapas anteriores.

1. Objetivo

A Fase IV acrescenta compressao aos arquivos de dados do sistema sem alterar o CRUD, os indices, a exclusao logica ou o relacionamento `Pedido N:N Produto`.

A compactacao foi implementada no nivel de arquivo, funcionando como um backup completo. O aplicativo gera um unico arquivo compactado contendo os arquivos utilizados pela aplicacao.

2. Arquivos compactados

Foram incluidos todos os arquivos `.db` de `data/`, abrangendo bases principais e indices:

- `clientes.db` e seus indices de hash extensivel.
- `produtos.db`, indice primario e indice B+ de ordenacao.
- `cupons.db` e seus indices.
- `pedidos.db`, indice primario e indices do relacionamento cliente-pedidos.
- `pedido_produto.db` e os indices B+ por pedido e por produto.

3. Empacotamento unico

A classe `Util.DataArchive` cria o arquivo original logico usado pelos dois algoritmos. Esse pacote armazena assinatura, quantidade de arquivos, nome relativo, tamanho e conteudo de cada arquivo.

```
[assinatura TPDB1]
[quantidade de arquivos]
[tamanho do nome][nome relativo][tamanho do arquivo][bytes do arquivo]
...
```

Com isso, Huffman e LZW comprimem exatamente a mesma entrada, e o resultado final continua sendo apenas um arquivo compactado por algoritmo.

4. Algoritmos implementados

4.1 Huffman

`Util.HuffmanCompressor` calcula a frequencia de cada byte, monta uma arvore de Huffman com fila de prioridade e grava um cabecalho com o tamanho original e as frequencias necessarias para reconstruir a arvore.

Arquivo gerado: `backups/fase4_huffman.huff`

4.2 LZW

`Util.LZWCompressor` inicia o dicionario com os 256 bytes possiveis e substitui sequencias repetidas por codigos de 16 bits. O dicionario foi limitado a 65.536 entradas para manter cada codigo em dois

bytes.

Arquivo gerado: backups/fase4_lzw.lzw

5. Formulario tecnico

1. Qual foi a taxa de compressao obtida com o algoritmo de Huffman?

Item	Valor
Tamanho do arquivo original	4461 bytes
Tamanho do arquivo comprimido	2925 bytes
Calculo da taxa	$1 - (2925 / 4461) = 0,3443 = 34,43\%$
Interpretacao	O Huffman reduziu o pacote em 34,43% . O resultado e positivo porque os arquivos binarios possuem muitos bytes repetidos, especialmente zeros, cabecalhos pequenos e estruturas de indice com campos vazios. A taxa nao e maior porque o backup e pequeno e o Huffman grava metadados de frequencia no proprio arquivo compactado.

2. Qual foi a taxa de compressao obtida com o algoritmo de LZW?

Item	Valor
Tamanho do arquivo original	4461 bytes
Tamanho do arquivo comprimido	2422 bytes
Calculo da taxa	$1 - (2422 / 4461) = 0,4571 = 45,71\%$
Interpretacao	O LZW reduziu o pacote em 45,71% , ficando melhor que Huffman nesta base. Isso ocorreu porque o pacote contem sequencias repetidas de bytes e padroes estruturais dos arquivos <code>.db</code> e dos indices.

3. Quais dificuldades surgiram ao implementar Huffman e LZW e como voce resolveu?

A primeira dificuldade foi atender ao requisito de gerar um unico arquivo compactado sem comprimir cada tabela separadamente. A solucao foi criar `DataArchive`, um formato simples de empacotamento que junta todos os `.db` em um unico fluxo antes da compressao.

No Huffman, o cuidado principal foi gravar metadados suficientes para descompactar depois. Para resolver isso, o arquivo compactado armazena o tamanho original e a tabela de frequencias dos bytes presentes, permitindo reconstruir a mesma arvore na leitura.

No LZW, a maior atenção foi manter o dicionário limitado para que cada código coubesse em 16 bits. O dicionário foi limitado a 65.536 entradas, e a descompressão trata o caso especial em que o código recebido é igual ao próximo índice do dicionário.

Por fim, para preservar a integridade, os dois algoritmos foram testados com descompressão imediata e comparação byte a byte com o pacote original.

6. Resultado da verificação

Algoritmo	Original	Comprimido	Taxa	Integridade
Huffman	4461 bytes	2925 bytes	34,43%	Verificado
LZW	4461 bytes	2422 bytes	45,71%	Verificado

A verificação foi feita pelo CLI `Util.BackupCli`, que gera os backups, descompacta em memória e compara byte a byte com o pacote original.

7. Exemplo de compressão

Os arquivos `backups/fase4_huffman.huff` e `backups/fase4_lzw.lzw` já são exemplos reais gerados pelo sistema. Eles não são legíveis diretamente em editor de texto porque armazenam dados binários compactados. O conteúdo deles corresponde ao pacote único criado a partir dos arquivos `.db` da pasta `data/`.

Arquivo compactado	Algoritmo	Entrada usada	Tamanho final
<code>backups/fase4_huffman.huff</code>	Huffman	Pacote único com 19 arquivos <code>.db</code>	2925 bytes
<code>backups/fase4_lzw.lzw</code>	LZW	Pacote único com 19 arquivos <code>.db</code>	2422 bytes

Para visualizar a ideia dos algoritmos, considere a sequência didática:

BANANA_BANANA

No Huffman, os símbolos mais frequentes recebem códigos menores. Nessa sequência, **A** e **N** aparecem mais vezes que **B** e **_**, então tenderiam a receber códigos binários mais curtos. O ganho vem da troca de bytes fixos por códigos de tamanho variável.

No LZW, o compressor cria entradas para sequências repetidas. Depois de ler partes como **BA**, **AN**, **NA** e **ANA**, a segunda ocorrência de **BANANA** pode ser representada por códigos de dicionário, reduzindo a repetição literal dos bytes.

8. Como executar

```
New-Item -ItemType Directory -Force out\classes | Out-Null
$sources = Get-ChildItem -Recurse -Filter *.java -Path Controller,DAO,Main,Model,Util,View |
ForEach-Object { $_.FullName }
javac --release 8 -d out\classes $sources
java -cp out\classes Util.BackupCli
```

Pela interface web, execute o servidor com `java -cp out\classes Main.App` e acesse `http://localhost:18080/compressao`.

9. Conclusao

A Fase IV foi implementada como backup completo em arquivo unico. A compressao atua apenas sobre uma copia empacotada dos dados e indices, entao o funcionamento das fases anteriores permanece preservado. Huffman e LZW foram implementados sem bibliotecas externas e com verificacao de integridade por descompressao.

Documento preparado para a entrega da Fase IV em 05/06/2026.